

Real-Time Weather Data Analysis for Predicting Renewable Energy Production – Tests

Magdalena Jeczeń, Aleksandra Kulczycka, Jan Pogłód,

Zofia Sasimowska, Gabriel Vogel

Test ID	Test Name	Objective	Steps	Expected Result	Actual Result
FT-01	Weather API connectivity	Verify that Open-Meteo API responds	Send GET request	Status 200, valid JSON	PASS: Status code: 200 JSON received. Keys: ['latitude', 'longitude', 'generationtime_ms', 'utc_offset_seconds', 'timezone', 'timezone_abbreviation', 'elevation', 'hourly_units', 'hourly']
FT-02	Electricity API connectivity	Check that ElectricityMaps returns electricity mix	Send GET with token	Status 200, JSON contains "data"	PASS: Status code: 200 JSON received. Keys: ['zone', 'temporalGranularity', 'unit', 'data']
FT-03	Weather JSON → DataFrame	Verify correct parsing	Convert JSON into DataFrame	DataFrame has correct columns Shape=(192,6)	PASS: df.shape = (192, 6) Img.1
FT-04	Electricity JSON → DataFrame	Verify correct parsing of electricity-mix JSON into a normalized DataFrame	1. Send GET request to ElectricityMaps API. 2. Extract electricity mix data from the response. 3. Convert mix data into a DataFrame and add timestamp metadata.	DataFrame has correct columns Index is of type datetime64[ns]. Shape=(12,3) (because datetime is an index)	PASS: df.shape = (12,3) (because datetime is an index) Img.2
FT-05	Timestamp parsing	Ensure timestamps parse correctly	Apply pd.to_datetime	Column dtype = datetime64	PASS — dtype printed: datetime64[ns, UTC]
FT-06	Parquet output	Verify Parquet file creation	Save DataFrame to .parquet	File exists	PASS — files created: File saved successfully: weather.parquet File saved successfully: electricity.parquet

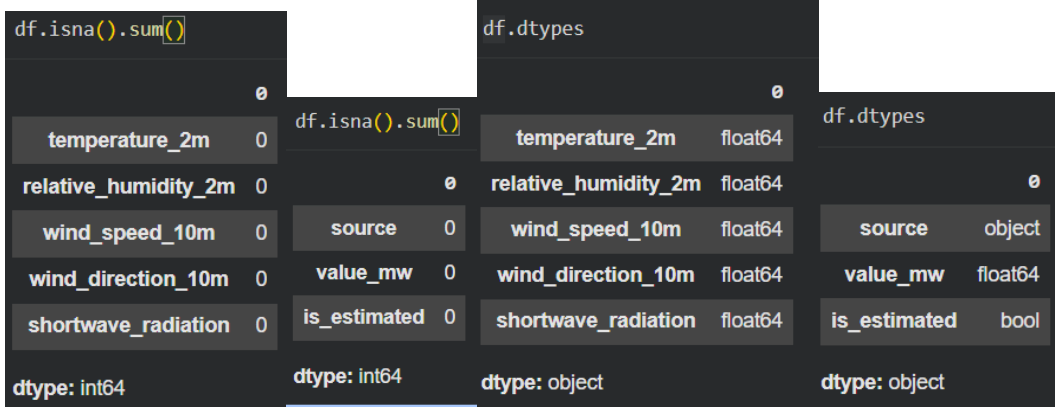
	time	temperature_2m	relative_humidity_2m	wind_speed_10m	wind_direction_10m	shortwave_radiation
0	2025-11-22T00:00	-1.4	89	6.6	279	0.0
1	2025-11-22T01:00	-2.1	93	5.5	281	0.0
2	2025-11-22T02:00	-2.2	93	5.8	277	0.0
3	2025-11-22T03:00	-2.4	92	5.1	262	0.0
4	2025-11-22T04:00	-2.8	95	3.2	243	0.0

Img.1

	source	value_mw	is_estimated
datetime			
2025-11-24 09:00:00+00:00	nuclear	NaN	True
2025-11-24 09:00:00+00:00	geothermal	NaN	True
2025-11-24 09:00:00+00:00	biomass	364.0	True
2025-11-24 09:00:00+00:00	coal	13784.0	True
2025-11-24 09:00:00+00:00	wind	2349.0	True

Img. 2

Test ID	Test Name	Objective	Steps	Expected Result	Actual Result
QT-01	Weather missing values	Check for missing data	df.isna().sum()	No missing values	PASS — all columns show 0 missing values Img. 3
QT-02	Electricity null → 0	Ensure nulls replaced	Check df.isna().sum()	Zero NaN	PASS — 0 NaN in value_mw Img. 4
QT-03	Type validation	Ensure correct dtypes	Check df.dtypes	Weather: numeric columns float64/int64, timestamp datetime64. Electricity: value_mw float64, source string, is_estimated bool, datetime64 index.	PASS — all columns dtypes as expected Img. 5 & Img. 6
QT-04	Timestamp completeness	Validate periodicity	Count records	Weather: 24/day	PASS — retrieved 192 records / 8 days = 24/ day
QT-05	Schema consistency	Ensure stable schema	Run pipeline twice	Identical schema	PASS — Schemas identical: True



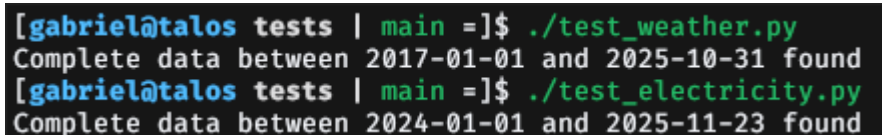
Img. 3

Img. 4

Img. 5

Img.6

The following tests were conducted using the scripts in the history/tests/ directory of the git repo. Both ran successfully:



Img. 7

Test ID	Objective	Steps	Expected Result	Actual Result
HT-01	Read historical data	Parse all JSON files	No parsing errors	PASS — everything could be read
HT-02	Verify number of data points	Compare array lengths with expected size	Number of data points matches time range covered by file	PASS — all files hold correct number of data points
HT-03	Verify data timestamps	Compare first timestamp listed in each file	First timestamp should match name of containing file	PASS — timestamps and file names match up
HT-04	Verify data types	Check JSON types of data values	Data should be numerical, sometimes null is allowed	PASS — data is saved as proper type